

MonthlyMuse

Technology Stack — detailed reference

AI-Powered Automated Message Generator for Monthly Posting · companion to the Master Blueprint (Sections 13–14) · October 2026

- FastAPI backend
- Next.js 14 frontend
- 16-table relational DB
- own NumPy NLP engine
- APScheduler worker
- 62 passing tests

1. Stack at a glance

Layer	Choice (installed version)	Why this choice	Blueprint alternative
Backend framework	FastAPI 0.115 + Uvicorn 0.30	Same language as the ML code; async; automatic OpenAPI docs; Pydantic validation on every route	Flask, Django REST, Node (would split the codebase)
Validation / schemas	Pydantic 2.9 + pydantic-settings 2.5	Typed request/response models, env-driven config (<code>MM_</code> prefix), .env support	marshmallow, class-validator
ORM	SQLAlchemy 2.0.35 (declarative)	Mature, explicit control over queries; Alembic 1.13 ready for versioned schema	SQLModel, Prisma (JS)
Database	SQLite default → PostgreSQL 15/16 + pgvector via <code>MM_DATABASE_URL</code>	Zero-setup local dev; schema is identical on Postgres; vectors stored as float32 BLOBs, cosine computed in NumPy	PostgreSQL + pgvector
NLP preprocessing	Own <code>app/ai/text.py</code> (normalise, tokenize, lemmas, keywords, n-gram Jaccard, banned-word/CTA checks)	Zero heavy deps, deterministic, CPU-only; blueprint's spaCy/YAKE kept as an optional extra	spaCy small model + YAKE
Embeddings	Own 384-d hashing embedder (default) · <code>sentence-transformers all-MiniLM-L6-v2</code> optional via <code>MM_EMBEDDER</code>	Deterministic, offline, instant startup; same 384 dimensionality so every formula is unchanged; swap to MiniLM without code changes	sentence-transformers all-MiniLM-L6-v2
Numerical / ML	NumPy 1.26 + SciPy 1.15 (pure Python algorithms)	Matrix similarity ($E \cdot H^T$, $E \cdot E^T$), logistic-regression tone classifier, chi-square, Wilson intervals, Monte-Carlo — no GPU needed	NumPy, scikit-learn, SciPy
Generation (LLM)	Adapter layer: <code>template</code> (default, offline) · OpenAI · Anthropic · Ollama — retry 1s/3s/9s + jitter, circuit breaker (5 fails/60 s), tolerant JSON parse, cost metering	One call yields six candidates; the template engine (~40 templates + slot banks) guarantees the demo works with zero API keys	Hosted small model (GPT-mini / Claude / Gemini Flash class), Ollama fallback
Scheduling	APScheduler 3.11 (in-process + standalone worker)	No extra infrastructure; cron 02:00 daily scan + 5-min interval; enough for hundreds of users	APScheduler; Celery + Redis when scaling
Email	Console backend (dev) · SMTP / Resend options (<code>MM_EMAIL_BACKEND</code>)	Free tiers, simple API; in-app notifications always stored in DB	Resend / SendGrid / SMTP (Mailhog in dev)
Auth	Argon2id (argon2-cffi 25) + JWT (PyJWT 2.15): 15-min access + 14-day refresh httpOnly cookie; signed ICS calendar token	Stateless, standard, secure when cookie flags are right	Auth0 / Clerk / Supabase Auth
Frontend	Next.js 14.2 App Router + TypeScript 5 , React 18	Industry standard, typed contracts, easy deploy, static + dynamic routes	React + Vite

UI toolkit	Tailwind CSS 3.4 with Section-12 design tokens + own component library (<code>components/ui.tsx</code>)	Fast, polished, accessible; tokens (indigo primary, chart palette, text scale, shadows) implemented as Tailwind theme — shadcn/ui CLI dropped to keep the dependency tree auditable	Tailwind + shadcn/ui (MUI, Chakra)
Data fetching	TanStack Query 5	Caching, polling (generation status every 1.5 s), invalidation after feedback, error toasts	SWR
Forms	React Hook Form 7 + Zod 3 (<code>@hookform/resolvers</code>)	Schema-validated 3-step Create form with draft autosave	Formik
Charts	Recharts 3	Declarative React charts incl. radar (compare mode), scatter (PCA embedding map), donut/bars/CI bars	Chart.js, Plotly
Testing	pytest 9 (62 tests) · tsc + ESLint + <code>next build</code> for the frontend	Covers golden maths values, pipeline, API journey, scheduler automation	pytest, Playwright, Vitest/RTL
DevOps	Docker + docker-compose + GitHub Actions CI (backend matrix 3.10/3.11, frontend build)	Reproducible setup and automated quality gate	same
Monitoring	JSON structured logs (<code>request_id</code>), <code>/admin/jobs</code> job-run audit, <code>/admin/health</code> , per-request latency + LLM cost metrics	Cheap visibility into failures	Structured logs, Sentry, admin jobs page

2. Architecture (blueprint Section 13)

FRONTEND — Next.js 14 + TypeScript + Tailwind

Dashboard · Create · Review/Compare · Calendar · Plans · History · Analytics · Settings — calls only the public API; no secrets in the browser

| HTTPS / JSON — JWT access token + refresh cookie ▼

BACKEND API — FastAPI

[Auth + RBAC] → [Validation] → [Routers (61 endpoints)] → [Services] → [uniform error / JSON-logging layer]



AI / NLP PIPELINE — pure NumPy (app/ai/)

preprocess + keywords → context + history retrieval → Thompson-sampled style arms → LLM adapter (retry, breaker, template fallback) → 384-d embeddings → cosine similarity → features $F, s = F \cdot w$ → MMR top-3 → Monte-Carlo confidence → explanation payload



DATABASE — SQLite (default) / PostgreSQL + pgvector

16 tables: users, user_preferences, audience_profiles, topics, occasions, monthly_plans, post_cycles, generation_requests, generated_messages, feedback_events, tone_stats, scheduled_posts, post_metrics, notifications, job_runs, analytics_daily



AUTOMATION — APScheduler worker

daily cycle scan → enqueue generation → notify → T-3/T-1 reminders → catch-up → plan extension · job_runs audit

Statelessness rule from the blueprint is respected: the API holds no in-memory user state, so it can run behind several instances; the worker shares the same code and database but starts with a different command (`python -m app.scheduler.runner`).

3. Backend in detail

3.1 Runtime & structure

```
backend/app/
├─ api/      auth, users, catalog, plans, cycles, messages, analytics, misc (61 routes)
├─ core/     config.py (MM_ env), errors.py, security.py, logging.py, rate_limit.py, runtime.py
├─ db/       session.py (engine, UTCDateTime, blob<->vector helpers), models.py (16 tables)
├─ schemas/  Pydantic request/response models
├─ ai/       text, embedder, similarity, features, scoring, mmr, confidence, arms,
│            tone, llm_adapters, prompt, templates, limits, pipeline, personalization
├─ services/ account, catalog, plan, generation, feedback, history, notification, analytics
└─ scheduler/ jobs.py, runner.py (APScheduler: cron 02:00 + interval 5 min)
```

- **Python 3.10.11**, installed with `pip install -e ".[dev]"` from `backend/pyproject.toml`.
- **App lifespan**: create tables → seed catalog (60 occasions, 20 topics, 40 templates) → train/load tone classifier → start in-process scheduler → serve.
- **Async where it matters**: generation runs in a background task (HTTP 202 + polling endpoint) so requests never block; LLM adapters use `httpx.AsyncClient`.

3.2 Configuration (pydantic-settings, prefix MM_)

Variable	Default	Purpose
MM_DATABASE_URL	sqlite:///./monthlymuse.db	Swap to postgresql+psycopg://... — schema unchanged
MM_LLM_PROVIDER	template	template openai anthropic ollama
MM_EMBEDDER	hash	hash (own 384-d) or sentence-transformers
MM_SCHEDULER_ENABLED / MM_SCHEDULER_TIMEZONE	true / Asia/Kolkata	Worker behaviour and reminder timezone
MM_SECRET_KEY	dev value	JWT signing — changed per environment, never committed
MM_GENERATIONS_PER_HOUR / MM_REQUESTS_PER_MINUTE	10 / 100	Sliding-window rate limits
MM_LLM_MONTHLY_BUDGET_USD	5.0	Cost guardrail fed by the PRICING table
MM_EMAIL_BACKEND	console	console smtp resend

3.3 Security

- **Argon2id** password hashing (argon2-cffi), tuned parameters, constant-time compare.
- **JWT access (15 min)** in memory + **refresh (14 days)** in an httpOnly, SameSite cookie; silent refresh in the API client hides 401s.

- Signed **ICS calendar token** so subscription URLs work without a login.
- Uniform error envelope `{error: {code, message, details, request_id}}`, CORS allow-list, per-user + global rate limits, structured JSON logs carrying `request_id`.

4. AI / NLP engine in detail

All algorithms are pure NumPy/SciPy and unit-tested against the blueprint's Section 17 golden values.

Stage	Implementation	Key parameters / guarantees
1. Preprocessing	<code>ai/text.py</code>	normalize → tokenize → lightweight lemma → keyword extraction (TF-IDF-ish scoring), n-gram Jaccard dedup, banned-word & CTA checks
2. Embeddings	<code>ai/embedder.py</code>	384-d feature hashing with signed weights, L2-normalised; optional MiniLM; vectors stored as float32 BLOBs
3. Similarity	<code>ai/similarity.py</code>	<code>cosine</code> , $E \cdot H^T$ history matrix, novelty = $1 - \max \cos$, PCA 2-D map for the UI
4. Features	<code>ai/features.py</code>	relevance, tone, personalization (e,u), novelty, length (z-score), history (Beta) → row $F \in [0,1]^6$
5. Scoring	<code>ai/scoring.py</code>	$s = F \cdot w$, default $w = (.30,.20,.20,.15,.10,.05)$; constrained gradient step ($\lambda=0.005$, $\beta=8$, $lr=0.01$) projected onto the simplex; reason-nudge from reject reasons
6. Diversity	<code>ai/mmr.py</code>	MMR $\lambda=0.7$ with greedy + exhaustive options and style-arm quotas so the top-3 differ
7. Confidence	<code>ai/confidence.py</code>	Dirichlet Monte-Carlo, 200 seeded samples (seed 42) → $P(\text{rank-1}) + \text{band}$; Wilson intervals for rates
8. Arms	<code>ai/arms.py</code>	Thompson sampling over tone × structure arms (Beta priors, 2 exploit + 1 explore), Laplace-smoothed tone suggestions
9. Tone	<code>ai/tone.py</code>	Softmax logistic regression trained on 800 labeled rows at startup, weights cached to <code>.npz</code> , confusion-matrix helper
10. Generation	<code>ai/llm_adapters.py</code>	OpenAI/Anthropic/Ollama/Template behind one interface; retry 1s/3s/9s + jitter, circuit breaker 5 fails/60 s, tolerant JSON parse, PRICING cost table
11. Personalization	<code>ai/personalization.py</code>	reward model → preference vector u (EMA), Beta arm stats, learned length stats, online weight updates, edit-distance attribution

Determinism: seeded RNG everywhere (Monte-Carlo, Thompson, training) — same inputs and weights always produce the same ranking, which is what makes the golden-value test suite possible.

5. Frontend in detail

- **Next.js 14.2 (App Router) + TypeScript 5** — 13 routes; client components only where interactivity demands it; production build $\approx$ 88 kB shared JS.
- **Tailwind CSS 3.4** implements the Section 12 design system: indigo primary scale, 7-color chart palette, type scale (display $\rightarrow$ small), radii, shadows, plus component classes (`card` , `btn` , `input` , `chip` , `badge`) and reduced-motion support.
- **TanStack Query 5** — one `http` wrapper (`lib/api.ts`) handles JSON, auth headers, silent refresh on 401, and typed errors; generation status polls every 1.5 s until done.
- **React Hook Form + Zod** — the 3-step Create form validates with a Zod schema shared by all steps, autosaves drafts every 5 s.
- **Recharts 3** — radar chart for compare mode, 2-D scatter for the embedding map, donut/bars/line/CI bars for analytics.
- **lucide-react** icons, **date-fns** date formatting.

Screen	Blueprint section	What it does
Dashboard	3.1	Next-action banner, this-month card, 12-cycle timeline, quick stats, activity feed
Create	3.2	3-step form, tone chips with Bayesian recommendation hints, occasion/keyword chips, “what the AI will use” panel
Review / Compare	3.3	Stage tracker, 3 ranked cards with score rings + confidence + similarity meters, why-ranked text, expandable feature bars, compare mode (radar + highlighted diffs), PCA embedding map, filtered-candidates disclosure, select/edit/copy/like/dislike/reject/regenerate
Calendar	3.4	Month grid with status chips, cycle modal (generate now / skip / reschedule), .ics download
Plans	3.7	List/create/pause/resume/delete Monthly Plans
History	3.5	Text + semantic search, filters, detail drawer with batch context, reuse
Analytics	3.6	Usage / Content / Quality tabs: donut, bars, lines, Wilson CI bars, chi-square panel, novelty & entropy
Settings	3.7	Voice, banned words, emoji, length, variety slider, pause learning, learned priorities + tone arms + reset, audiences CRUD, export/delete

6. Data layer

- **16 tables** per blueprint Section 9 (listed in §2 above); portable string enums instead of native PG enums so SQLite and Postgres share one schema.
- **UTC everywhere**: a custom `UTCDateTime` TypeDecorator normalises SQLite's naive datetimes back to aware UTC — no naive/aware mixing bugs.
- **Vectors** as float32 BLOBs (`vec_to_blob/blob_to_vec`); cosine similarity computed in NumPy — identical results on SQLite and Postgres, pgvector-ready for V2 scale.

- **JSON columns** where the blueprint allows them (tone_counts, per-request stage output) keep reads cheap without a schema change.
- Alembic 1.13 is installed and the metadata is importable, so `alembic revision --autogenerate` works out of the box for the Postgres deployment.

7. Automation (worker)

- **APScheduler 3.11**: daily cron 02:00 (timezone `MM_SCHEDULER_TIMEZONE`) + 5-minute interval trigger; runs in-process by default, or standalone via `python -m app.scheduler.runner` (compose does the latter).
- Jobs: `prepare_due_cycles` (idempotent — creates next month's cycle + generation request), `send_reminders` (T-3, T-1, and NEEDS_ATTENTION nudges), `catch_up` (missed runs), `extend_plans` (auto-extend the 12-cycle horizon), all recorded in `job_runs` for the admin page.
- Cycle state machine: `PLANNED` → `GENERATING` → `AWAITING_REVIEW` → `APPROVED` → `SCHEDULED` → `PUBLISHED` (+ `SKIPPED` / `FAILED`).

8. Testing, CI, DevOps

Suite	Contents
<code>test_math.py</code> (35)	Section 17 golden values: feature/score vector, gradient steps ($w_N \rightarrow 0.161$, $w_P \rightarrow 0.195$, $w_L \rightarrow 0.094$), MMR order [A,C,F], Wilson CI (3,10)=(0.108,0.603), choice probabilities $\approx [0.46, 0.34, 0.20]$
<code>test_pipeline.py</code> (12)	Hard constraints, dedup, arm-aware MMR, confidence seeding, stage payload
<code>test_api.py</code>	End-to-end journey: register → plan → generate (202 + poll) → feedback → approve → analytics
<code>test_scheduler.py</code> (4)	Reminder days, idempotent preparation, catch-up, plan extension
Frontend gate	<code>tsc --noEmit</code> + ESLint + <code>next build</code> (15 routes, 0 warnings)

CI (`.github/workflows/ci.yml`): backend pytest on Python 3.10 & 3.11; frontend typecheck + production build on Node 20 — every push/PR. **Compose** (`docker-compose.yml`): postgres:16 + api + worker + frontend, health-checked, one command start.

9. Deviations from the blueprint (and why)

Blueprint recommendation	What we shipped	Reason
PostgreSQL + pgvector	SQLite default, Postgres-ready	No Docker/Postgres on the target machine; one env var switches, schema & tests identical (chosen with the product owner)
sentence-transformers (384-d)	Own hashing embedder by default, MiniLM optional	Instant startup, offline, deterministic; same dimensionality so every formula/test is unchanged (chosen with the product owner)
spaCy + YAKE preprocessing	Own <code>text.py</code> (spaCy as an optional extra)	Heavy native deps; the needed operations are small and testable in isolation
shadcn/ui	Own component library on Tailwind tokens	Fewer moving parts, full control of the Section 12 tokens, no CLI-generated code drift
Hosted LLM in the hot path	Template engine as default provider	Zero-cost, always-available demo; hosted adapters keep retry/breaker/cost metering for production

10. Runtime requirements

- **Dev machine:** Python 3.10+ (tested 3.10.11), Node 18+ (tested 22.19 / npm 10.9). No Docker or PostgreSQL needed for the default SQLite mode.
- **Production:** any Docker host — `docker compose up --build` (Postgres 16, API :8000, worker, frontend :3000); secrets via environment, never in Git.
- **Scale path:** swap DB URL → Postgres + pgvector, run multiple stateless API instances, move the worker to its own container (already supported), and enable the MiniLM embedder for stronger semantics — no code changes required.